```
@{
    Layout = "_Layout";
}
```

Any code in the _ViewStart.cshtml file runs before the view executes. Note that _ViewStart .cshtml runs only for Razor Page views; it doesn't run for layouts or partial views. Also note that the names for these special Razor files are enforced and can't be changed by conventions.

WARNING You must use the names _ViewStart.cshtml and _ViewImports.cshtml for the Razor engine to locate and execute them correctly. To apply them to all your app's pages, add them to the root of the Pages folder, not to the Shared subfolder.

You can specify additional _ViewStart.cshtml or _ViewImports.cshtml files to run for a subset of your views by including them in a subfolder in Pages. The files in the subfolders run after the files in the root Pages folder.

Partial views, layouts, and AJAX

This chapter describes using Razor to render full HTML pages server-side, which are then sent to the user's browser in traditional web apps. A common alternative approach when building web apps is to use a JavaScript client-side framework to build an SPA, which renders the HTML client-side in the browser.

One of the technologies SPAs typically use is AJAX (Asynchronous JavaScript and XML), in which the browser sends requests to your ASP.NET Core app without reloading a whole new page. It's also possible to use AJAX requests with apps that use server-side rendering. To do so, you'd use JavaScript to request an update for part of a page.

If you want to use AJAX with an app that uses Razor, you should consider making extensive use of partial views. Then you can expose these via additional Razor Page handlers, as shown in this article: http://mng.bz/vzB1. Using AJAX can reduce the overall amount of data that needs to be sent back and forth between the browser and your app, and it can make your app feel smoother and more responsive, as it requires fewer full-page loads. But using AJAX with Razor can add complexity, especially for larger apps. If you foresee yourself making extensive use of AJAX to